BLOCKCHAIN Experiment No: 6

Manas Patil
D20A/24

Aim : Creation and Deployment of Smart Contract and Transactions using Solidity and Remix IDE.

Tasks to be Performed :

1. Open Remix IDE.
2. Under Workspaces, open the contracts folder.
3. Open the Ballot.sol contract.
4. Understand the Ballot.sol contract.
5. Deploy the contract by changing the proposal name from bytes32 to string.

Theory :

1. Relevance of require Statements in Solidity Programs

In Solidity, the require statement acts as a guard condition within functions. It ensures that only valid inputs or authorized users can execute certain parts of the code. If the condition inside require is not satisfied, the function execution stops immediately, and all state changes made during that transaction are reverted to their original state.

In a voting smart contract, require can be used to check whether the person calling the function has the right to vote, whether a voter has already voted, and whether the function caller is the chairperson before granting voting rights. Thus, require statements enforce security, correctness, and reliability in smart contracts and can include error messages for easier debugging.

2. Keywords: mapping, storage and memory

Mapping: A mapping is a special data structure in Solidity that links keys to values, similar to a hash table. In the Ballot contract, mapping(address => Voter) public voters associates each Ethereum address with its Voter structure. Mappings are efficient for lookups but do not have a length property and cannot be iterated over directly.

Storage: Storage refers to the permanent data area of the contract stored on the Ethereum blockchain. Contract-level state variables are stored in storage by default. Storage persists across transactions, but writing to it consumes gas.

Memory: Memory is temporary storage used during the execution of a function. Data stored in memory is discarded after the function call ends. It is commonly used for temporary variables and function arguments and is cheaper than permanent storage.

3. Why bytes32 Instead of string?

bytes32 is a fixed-size type that stores exactly 32 bytes. It provides simple and efficient storage and is suitable for short proposal names, although it limits the data to 32 bytes. string is dynamically sized and can store text of variable length, making it more readable and user-friendly, but it requires more complex handling and may consume more gas. In this experiment, bytes32 is changed to string to make proposal names easier to enter and display.

Tools and Libraries Used

- Remix IDE
- Solidity
- Remix VM (for local deployment and testing)
- Ethers (used by Remix for Ethereum interaction)

Procedure

The following procedure was followed to create, compile, deploy and test the Ballot voting smart contract. The original Ballot contract was modified so that proposal names use string instead of bytes32.

Program: Modified Ballot.sol

```
// SPDX-License-Identifier: GPL-3.0

pragma solidity >=0.7.0 <0.9.0;

contract Ballot {

struct Voter {

uint weight;

bool voted;

address delegate;

uint vote;

}

struct Proposal {

string name;

uint voteCount;

}

address public chairperson;

mapping(address => Voter) public voters;

Proposal[] public proposals;

constructor(string[] memory
proposalNames) {

chairperson = msg.sender;

voters[chairperson].weight = 1;

for (uint i = 0; i < proposalNames.length;
i++) {

proposals.push(

Proposal({

name: proposalNames[i],

voteCount: 0

})
```

```
);

}}

function giveRightToVote(address voter)
external {

require(

msg.sender == chairperson,

"Only chairperson can give right to vote."

);

require(

!voters[voter].voted,

"The voter already voted."

);

require(

voters[voter].weight == 0,

"Voter already has the right to vote."

);

voters[voter].weight = 1;

}

function delegate(address to) external {

Voter storage sender =
voters[msg.sender];

require(sender.weight != 0, "You have no
right to vote");

require(!sender.voted, "You already
voted.");

require(to != msg.sender, "Self-delegation
is disallowed.");

while (voters[to].delegate != address(0)) {

to = voters[to].delegate;

require(to != msg.sender, "Found loop in
delegation.");

}

Voter storage delegate_ = voters[to];

require(delegate_.weight >= 1);

sender.voted = true;

sender.delegate = to;

if (delegate_.voted) {

proposals[delegate_.vote].voteCount +=
sender.weight;

} else {

delegate_.weight += sender.weight;

}}

function vote(uint proposal) external {

Voter storage sender =
voters[msg.sender];

require(sender.weight != 0, "Has no right
to vote");

require(!sender.voted, "Already voted.");

sender.voted = true;

sender.vote = proposal;

proposals[proposal].voteCount +=
sender.weight;

}

function winningProposal()
```

```
    public
    view
    returns (uint winningProposal_)
    {
    uint winningVoteCount = 0;
    for (uint p = 0; p < proposals.length; p++)
    {
    if (proposals[p].voteCount >
    winningVoteCount) {
    winningVoteCount =
    proposals[p].voteCount;
        winningProposal_ = p;
        }}}
    function winnerName()
        external
        view
        returns (string memory winnerName_)
        {
        winnerName_ =
        proposals[winningProposal()].name;
        }}
```

Step-by-Step Execution with Screenshots

Step 1: Open Remix IDE

Remix IDE was opened in the web browser. Remix provides the environment required to write, compile, deploy and interact with Solidity smart contract**s.**

Step 2: Create/Open Ballot.sol

Inside the contracts workspace, the Ballot.sol file was created/opened. The Ballot contract implements voting, delegation and winner calculation.

Step 3: Change Proposal Name from bytes32 to string

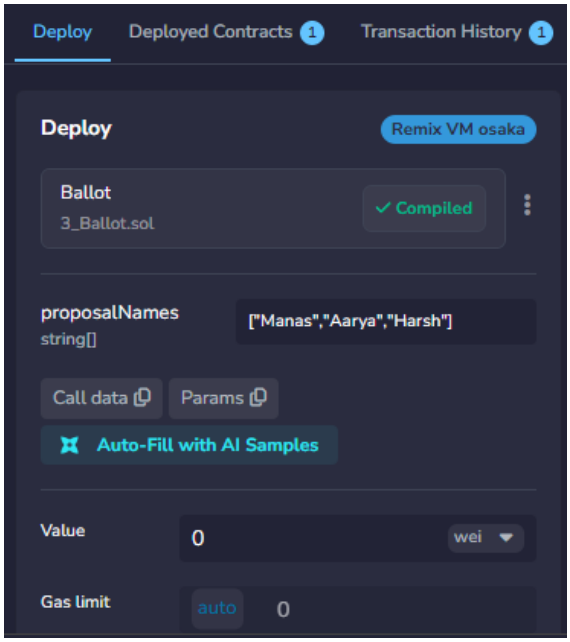
The proposal name type was changed from bytes32 to string. The constructor was changed from bytes32[] memory to string[] memory, and winnerName() was updated to return string memory. This allows proposal names to be entered directly as readable text.

Step 4: Compile Ballot.sol

The modified Ballot.sol program was compiled successfully in Remix. Successful compilation produces the contract ABI and bytecode required for deployment.

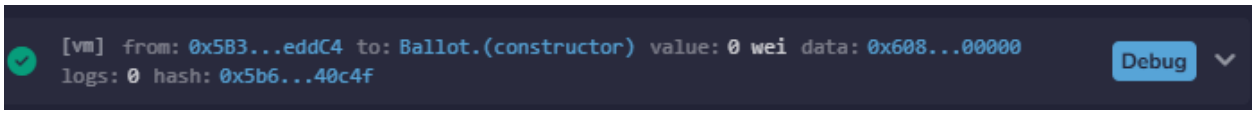
Step 5: Enter Proposal Names and Prepare Deployment

The deployment form was populated with the proposal names shown in the screenshot: Manas, Aarya and Harsh. The constructor accepts these values as a string array.



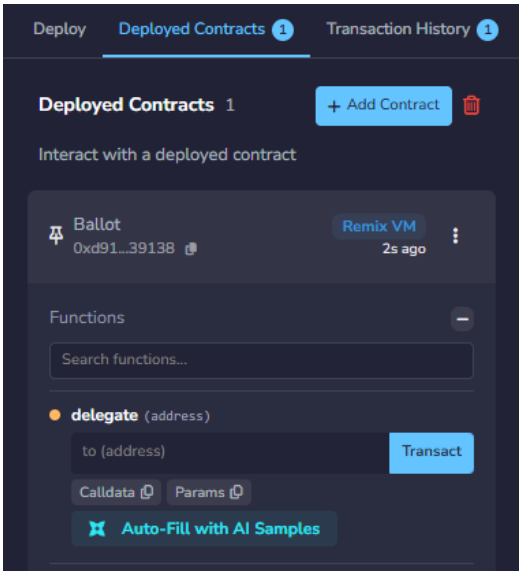
Step 6: Deploy the Ballot Smart Contract

The Ballot contract was deployed using Remix VM. The deployment transaction was submitted successfully and a deployed contract instance became available.



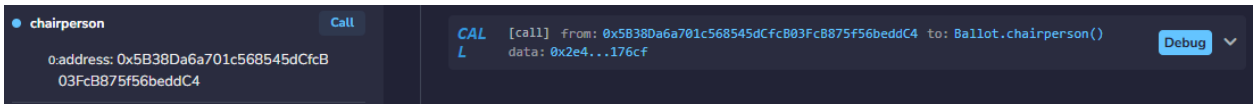
Step 7: Open the Deployed Contract

After deployment, the Ballot contract was available under Deployed Contracts. Its functions, including delegate, vote, giveRightToVote, proposals, voters, winningProposal and winnerName, could be interacted with.



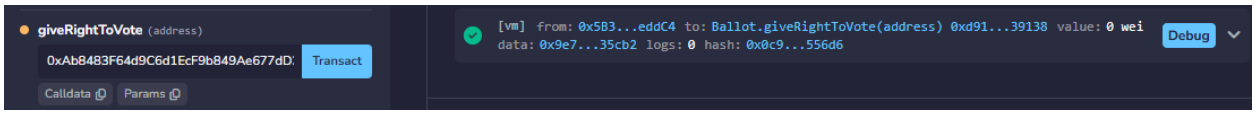
Step 8: Check the Chairperson

The chairperson() function was called to identify the account that deployed the contract. The deploying account becomes the chairperson through chairperson = msg.sender in the constructor.



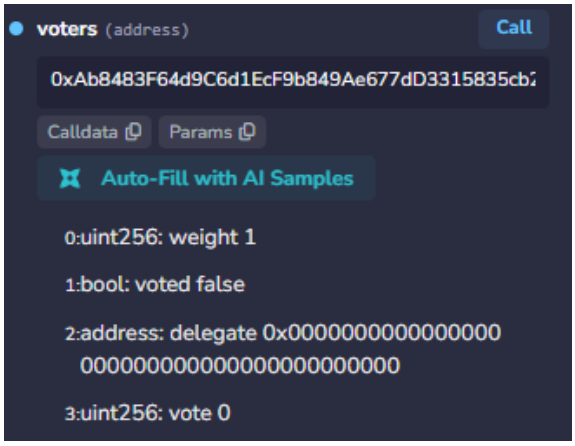
Step 9: Give Another Account the Right to Vote

The chairperson account was used to call giveRightToVote() with another account's address. This grants that voter a voting weight of 1 when the required conditions are satisfied.



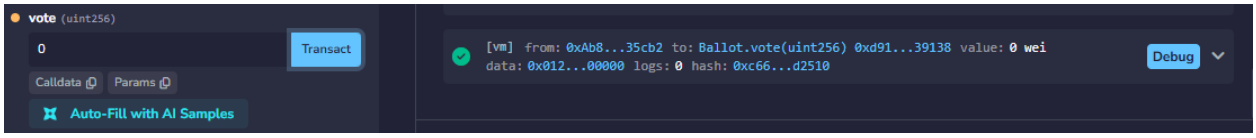
Step 10: Check Voter Information

The voters mapping was queried using the voter's address. The captured output shows weight 1, voted false, a zero delegate address and vote index 0, representing the voter's state before casting or delegating the vote.



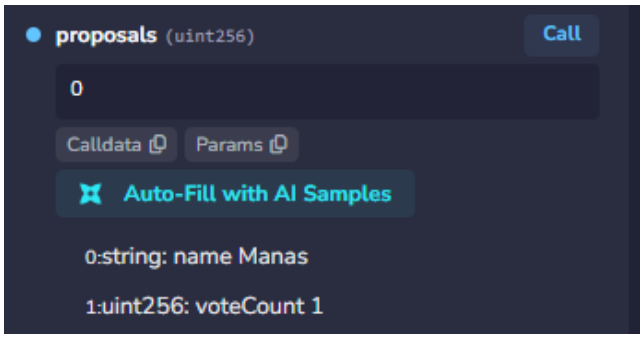
Step 11: Cast a Vote

The authorized voter account was selected and vote(0) was called. Proposal index 0 corresponds to Manas. The transaction was successfully recorded by the Ballot contract.



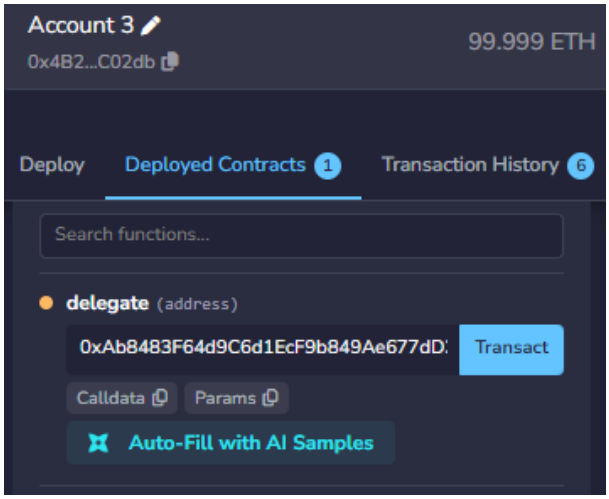
Step 12: Verify the Proposal Vote Count

The proposals array was queried for proposal index 0. The output shows name Manas and voteCount 1, confirming that the direct vote was recorded.



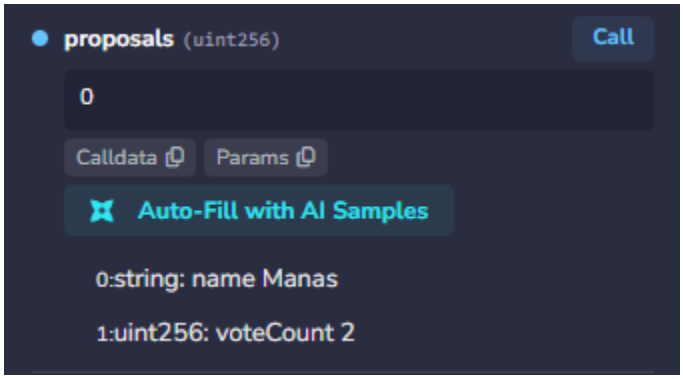
Step 13: Delegate a Vote

Another authorized account (Account 3 in the execution) was selected and delegate() was called with the address of the voter/delegate account. The delegation transaction was submitted successfully.



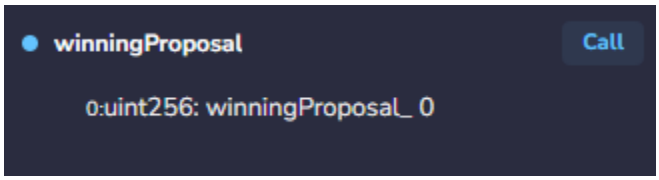
Step 14: Verify the Effect of Delegation

The proposal information was queried again. The vote count for Manas increased from 1 to 2, demonstrating the effect of the delegated vote on the proposal already selected by the delegate.



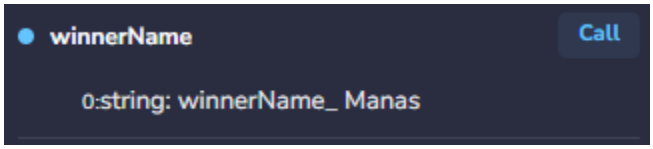
Step 15: Find the Winning Proposal

The winningProposal() view function was called. It returned proposal index 0. Since index 0 corresponds to Manas and Manas has the highest vote count, Manas is the winning proposal.



Step 16: Find the Winner's Name

Finally, winnerName() was called. It returned Manas, confirming the name of the winning proposal.



Output :

The Ballot smart contract was successfully compiled and deployed in Remix VM. Proposal names were entered as strings: Manas, Aarya and Harsh. Voting rights were granted to another account, a vote was cast for proposal index 0, and delegation was performed. The vote count of Manas increased to 2 after delegation. The winningProposal() function returned 0 and winnerName() returned Manas.

- Contract compilation completed successfully.
- Ballot contract deployed successfully using Remix VM.
- Proposal 0: Manas — final vote count: 2.
- winningProposal() returned 0.
- winnerName() returned Manas.

Result :

The Ballot smart contract was successfully created, compiled, deployed and tested using Solidity and Remix IDE. The voting and delegation transactions were executed successfully, and the winning proposal was identified.

Conclusion :

In this experiment, a Voting/Ballot smart contract was deployed using Solidity on the Remix IDE. The concepts of require statements, mapping, and data location specifiers such as storage and memory were explored to understand their role in security, efficiency and correctness in smart contracts. The difference between bytes32 and string for proposal names was also studied, highlighting the trade-off between gas efficiency and readability. Overall, the experiment provided practical insights into the design and deployment of voting contracts on the blockchain.